

AdroitCOM (.NET Agent Server interface) usage and guidelines

Introduction

The AdroitCOM interface was designed as a COM wrapper for as_api to allow managed applications (.NET) to communicate with the Adroit Agent Server.

Disclaimer

As a rule this interface is designed to be used exclusively as an internal api for software provided and maintained by Adroit Technologies. As a result this interface may be altered without warning possibly hindering the functionality of any third party applications making use of this interface directly.

Adroit Technologies also takes no responsibility whatsoever for any adverse effects induced through incorrect usage of this interface.

Note that certain methods listed hereon are explicitly stated as being for internal use only but any public methods not listed here but exposed by the interface are to be assumed to be for internal use and not for third party consumption.

Architecture

The AdroitCOM interface comprises essentially 3 assemblies:

- 1) AdroitCOM.dll
This is the COM wrapper for as_api and is registered in COM
- 2) Interop.AdroitCOMLib.dll
This is the Interop library which facilitates accessing the COM library via a managed (.NET) application. This is the assembly that is referenced by any managed application making use of this interface.
- 3) as_api.dll
This assembly is shipped as part of the Adroit install and facilitates access to the Agent Server via unmanaged applications.

As the AdroitCOM is a registered COM library, only one instance can be installed on a machine at any given time and care should be taken not to override any installed copy of this file as part of a normal Adroit / SmartUI installation. The Interop library (Interop.AdroitCOMLib.dll), which would be directly referenced by the managed application, would be incorporated and shipped as part of the third party application.

Functionality

Assume for all examples the following variables:

```
Interop.AdroitCOMLib.Adroit _adroit;  
Object _errorString;  
bool _error;  
Object _returnObject;
```

Connectivity

1) Connect

void Connect([string](#) bstrComputer, [string](#) bstrServer, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

This method opens a connection to the designated agent server. Passing empty strings for the machine and agent server names will connect to the local agent server.

Note: This will consume a remote client license if connecting to a remote machine.

2) ConnectEx

void ConnectEx([string](#) bstrComputer, [string](#) bstrServer, [string](#) bstrPassword, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

This method is only used internally to the SmartUI server and is not applicable to third party applications.

3) Disconnect

void Disconnect(ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Any connection made by way of the Connect method above is required to be closed through the Disconnect method.

General Operations

1) GetApiPath

void GetApiPath(ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Used internally to establish the path of the Adroit installation

2) GetServerList

void GetServerList(ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

This method returns a list of agent servers that are broadcasting on the network

3) GetVersion

void GetVersion(ref [object](#) pvariantResult)

Used internally to determine the version of as_api

4) SetTimeout

void SetTimeout([int](#) timeout, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Used internally to set the timeout applied to various function calls

Agent Operations

1) CreateAgent

void CreateAgent([string](#) bstrAgent, [string](#) bstrAgentType, [string](#) bstrAgentDescription, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Used to create new agents of a given type

2) GetSlotInfo

void GetSlotInfo([string](#) bstrAgent, [bool](#) variantboolIncludeHeaderSlots, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Returns an array detailing the slot information for a given agent (including the slot name and data type)

3) GetSlotNames

void GetSlotNames([string](#) bstrAgent, [bool](#) variantboolIncludeHeaderSlots, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Similar to the previous call but only returns a simple list of the slots for a given agent

4) Join

void Join([string](#) bstrGroup, [string](#) bstrAgents, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Allows a list of agents (comma delimited) to be added to a user defined group

5) Leave

void Leave([string](#) bstrGroup, [string](#) bstrAgents, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Removes a list of agents (comma delimited) from a user defined group

6) RemoveAgent

void RemoveAgent([string](#) bstrAgent, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Deletes a given agent. This method will fail if the agent has any outstanding memberships

Slot (Tag) Operations

1) GetSlot

void GetSlot([string](#) bstrAgent, [string](#) bstrSlot, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Basic operation to read the value of a given tag

2) GetSlotBulk

void GetSlotBulk([string](#) bstrTagNames, ref [object](#) pvariantValues, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Same as previous operation but allows reading of many tags in a single call

3) PutSlot

void PutSlot([string](#) bstrAgent, [string](#) bstrSlot, [object](#) variantValue, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Allows writing a value to a specified tag

4) PutSlotVariant

void PutSlotVariant([string](#) bstrAgent, [string](#) bstrSlot, [object](#) variantValue, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Enhanced version of the previous method honouring the data type passed as is. This is the preferred method.

5) PutSlotWithTime

void PutSlotWithTime([string](#) bstrAgent, [string](#) bstrSlot, [object](#) variantValue, [System.DateTime](#) dt, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Historical putslot method allowing a historical value to be inserted into the logging system

6) Subscribe

void Subscribe([string](#) bstrAgent, [string](#) bstrSlot, ref [int](#) plongSubscriptionIndex, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Obsolete, use bulk equivalent

7) SubscribeBulk

void SubscribeBulk([string](#) bstrTagNames, ref [string](#) arrayLongSubscriptionIndexes, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Allows for one or more tags (semi-colon delimited and terminated string) to be subscribed to in one call
example string: "systeminfo.second;" for single tag or "a1.value;a2.value;" for multiple

8) Unsubscribe

void Unsubscribe([int](#) longSubscriptionIndex, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Obsolete, use bulk equivalent

9) UnsubscribeBulk

void UnsubscribeBulk([string](#) bstrSubsIds, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Removes active subscriptions based on semi-colon delimited and terminated string of subscription ids as returned in the SubscribeBulk call.

10) AuditEntry

void AuditEntry([int](#) dwCategory, [string](#) szAction, [string](#) szObjectName, [string](#) szSlotName, [string](#) szObjectValue, [string](#) szRemarks, [string](#) szUsername, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Used internally to create Audit entries with custom parameters

Logging

1) LogTag

void LogTag([string](#) bstrAgent, [string](#) bstrSlot, [int](#) plongPeriod, [int](#) plongSet, [int](#) plongRate, [int](#) plongCompression, [string](#) bstrLogFile, [bool](#) pvariantboolStart, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Creates a datalog with the given parameters

Note that contrary to other operations like this where an explicit removal method exists, the way to remove a log is to remove the log agent as with any other agent.

Scanning

1) ScanTag

void ScanTag([string](#) bstrAgent, [string](#) bstrSlot, [string](#) bstrDeviceName, [string](#) bstrScanAddress, [int](#) plongScanRate, [int](#) plongDeadband, [bool](#) pvariantboolOutputenabled, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Scans a tag based on the given parameters (legacy method)

2) ScanTagEx

void ScanTagEx([string](#) bstrAgent, [string](#) bstrSlot, [string](#) bstrDeviceName, [string](#) bstrScanAddress, [int](#) plongScanRate, [int](#) plongDeadband, [bool](#) pvariantboolReadenabled, [bool](#) pvariantboolOutputenabled, [bool](#) pvariantboolOutputinitialize, [bool](#) pvariantboolDemandscan, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage);

Current method of adding scan points incorporating all scan modifiers.

3) UnScanTag

void UnScanTag([string](#) bstrAgent, [string](#) bstrSlot, [string](#) bstrDeviceName, [string](#) bstrScanAddress, [int](#) plongScanRate, [int](#) plongDeadband, [bool](#) pvariantboolOutputenabled, [int](#) plongUnscanKey, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Unscans a tag as per the parameters matching those the original scan was created with. Note that this function requires a key which is not made public and thus this function is only for internal use.

Alarming

1) AddAlarmRoute

void AddAlarmRoute([string](#) bstrAlarmAgent, [string](#) bstrAlarmLists, [string](#) bstrAudioDevices, [string](#) bstrOutputs, [string](#) bstrEventLogs, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Adds a new route to a given Alarm agent with the given parameters

2) AddAlarmType

void AddAlarmType([string](#) bstrAlarmAgent, [string](#) bstrAlarmTypeName, [string](#) bstrAgentType, [int](#) plongStatusMask, [string](#) bstrSlots, [bool](#) pvariantboolAlarmOffState, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Adds a new alarm type to a given Alarm agent with the given parameters

3) AlarmTag

void AlarmTag([string](#) bstrAgent, [string](#) bstrAlarmAgent, [string](#) bstrAlarmType, [int](#) plongRoute, [bool](#) pvariantboolAlarmAck, [int](#) plongAlarmTypeEnum, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Alarms a specified agent to a given Alarm agent with the supplied parameters

4) AlarmTagEx

void AlarmTagEx([string](#) bstrAgent, [string](#) bstrAlarmAgent, [string](#) bstrAlarmType, [int](#) plongRoute, [bool](#) pvariantboolAlarmAck, [int](#) plongPriority, [int](#) plongDelay, [string](#) bstrCondition, [System.DateTime](#) dtInhibitedUntil, [int](#) plongAlarmTypeEnum, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Same as the previous function with the addition of the condition, inhibit until and alarm delay parameters

5) ChangeAlarmRoute

void ChangeAlarmRoute([string](#) bstrAlarmAgent, [int](#) plongRoute, [string](#) bstrAlarmLists, [string](#) bstrAudioDevices, [string](#) bstrOutputs, [string](#) bstrEventLogs, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Allows the modification of an existing alarm route

Subscription Handling

Subscription handling is done through a callback method which is registered with the AdroitCOM object instance. This callback differs in structure from Adroit 10.0.5.1 onwards but both structures are shown herein.

The newer form of the callback is defined as follows:

```
void SubscribeCallBack(ref object subscriptionId, ref object subscriptionValue, ref object subscriptionStatus, ref object timeStamp, ref object pureStatusUpdate, ref object sequenceNumber)
```

Binding to the subscription callback is done similarly to binding to any event handler:

```
_adroit.SubscriptionUpdate += SubscribeCallBack
```

The legacy form of the callback is defined similarly although is missing the sequence number which is an essential part of current methodologies around handling asynchronous subscription updates

```
void SubscribeCallBack(ref object subscriptionId, ref object subscriptionValue, ref object subscriptionStatus, ref object timeStamp, ref object pureStatusUpdate)
```

Historical Data

1) GetHistoricalData

void GetHistoricalData([string](#) bstrAgent, [string](#) bstrSlot, [System.DateTime](#) dateStartTime, [System.DateTime](#) dateEndTime, [uint](#) ulongSampleRate, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Retrieves historical data for a given tag based on the supplied parameters

2) GetHistoricalDataEx

void GetHistoricalDataEx([string](#) bstrAgent, [string](#) bstrSlot, [System.DateTime](#) dateStartTime, [System.DateTime](#) dateEndTime, [int](#) longSamples, ref [object](#) pvariantResult, ref [bool](#) pvariantboolError, ref [object](#) pvariantErrorMessage)

Same as the previous function except that only actual data points are returned and no interpolated values are given.